Progress since your original brief

Decentralized Knowledge Marketplace

Status report and completion plan | 11 October 2026

Overall status: partially complete. The core application and many safety controls are implemented. The broader payment experience you requested is still unfinished. It would be inaccurate to say that only testing remains.

What this report covers

Your original architecture-review brief, the subsequent frontend/account and marketplace work, payment improvements, Transak sandbox integration, and the latest provider-independent blockchain receipt contract. The report separates existing foundations from changes made during this work.

Current position	Meaning
Core application	Welcome/authentication, marketplace, minting, search, auctions, protected content, reviews and account tools have implementation and targeted test evidence.
Payments	Existing token purchases and separate provider sandbox entry points exist. A single complete fiat or cross-chain purchase-to-payout flow is not yet connected.
Blockchain recording	New receipt registry source and local tests are complete. Its deployment, backend recording worker and entitlement connection are not complete.
Delivery scope	A local college demonstration. No new public deployment, mainnet payment or real-money checkout was performed in this work.

Evidence boundary

Repository snapshot: branch **codex/dkm-payment-persistence**, latest recorded commit **6612caf** (10 October 2026). Work is in existing draft PR #1; it has not been merged. This report consolidates prior verification and repository records; it is not a new full test run.

Read the report as: implemented = code exists; locally tested = specific checks passed; provider verified = the stated sandbox API operation succeeded; end-to-end verified = the whole buyer/seller journey succeeded. These are different claims.

Your requirements and the architecture

What you asked the project to do

Application flow. Show a welcome page first, then login/sign-up. Expose marketplace, creation, search, auctions and account functions only after authentication.

Usable marketplace. Connect backend features to the frontend, provide readable asset information, protect other users' accounts and avoid exposing raw file IDs or hashes as the normal product description.

Flexible buyer payments. Allow supported cryptocurrencies and fiat methods such as cards or UPI through suitable providers, with clear final charges.

Seller preferences. Support the seller's agreed pricing/payout currency; do not force every payment to route through Amoy merely because the original demo used it.

Blockchain evidence. Record verified purchases/payment outcomes on-chain and connect them safely to access rights.

College demonstration. Run locally through the actual UI and backend. External hosted provider checkout is acceptable. Public hosting is not the current objective.

Foundations that already existed at the original review

Next.js/React with wagmi and RainbowKit; FastAPI with asyncpg; PostgreSQL/pgvector and 384-dimensional embeddings; Pinata/IPFS; BLAKE3; LLM enrichment; semantic and graph/QUBO reranking; Solidity knowledge assets, marketplace, auctions and royalties. The initial payment service was mock-based and held orders in memory.

Architecture preserved and strengthened

The UI calls FastAPI for accounts, content and provider operations, and uses the wallet for contract transactions. PostgreSQL stores account, asset and payment state. New files are encrypted before IPFS upload. Provider success, blockchain confirmation and access entitlement are separate states.

Ownership rule: the creator keeps the NFT. A buyer purchases an on-chain access/license entitlement. This project does not currently transfer the NFT to each buyer.

Signup data: application users and sessions are stored in PostgreSQL, used through the project's Docker database setup. Passwords are salted hashes, not plaintext. Google authentication is optional configuration on top of the same account system.

Completed: UI, accounts and content

Frontend and identity

Welcome and protected navigation. Added the welcome-first flow, separate login/sign-up screens and session-gated application pages. Authentication screens no longer show marketplace/search/mint/dashboard controls.

Account security. Added password authentication, hashed session storage, HttpOnly cookies, logout, account-scoped records, origin/CSRF protection and authentication rate limiting.

Google sign-in. Implemented authorization-code login with state and PKCE. Owner OAuth configuration and interactive acceptance remain; this is not a confirmed working Google login today.

Wallet binding. Added signed wallet challenges, expiry and replay protection. A connected wallet alone is not accepted as proof that it belongs to the logged-in account.

Feature access. Connected marketplace, mint, search, auctions, account, funding, dashboard and status views. Asset descriptions use readable fields rather than presenting internal file IDs/CIDs/hashes as the primary information.

Asset pipeline and protected access

Upload and mint linkage. Fixed multipart metadata binding and the returned/stored vector-ID mismatch. Token linking checks the authenticated creator and on-chain asset details. Added recovery of unlinked uploads.

Duplicate protection. Moved duplicate checks to persistent content-hash records with concurrency control, rather than process-local memory alone.

Encrypted content. Added AES-GCM encryption for new files, wrapped file keys, integrity checks and authenticated downloads that verify creator or buyer access. Previously public IPFS content cannot be made private retroactively.

Search and reviews. Added usable search/filter controls and purchaser-only ratings/reviews, self-review rejection, aggregates and pagination.

Account activity. Added saved payment/order visibility and bounded fixed-price transaction activity. Complete indexed auction history and net lifetime earnings are still open.

These changes preserve the existing AI/search architecture. They do not establish that every backend function has passed a live, interactive UI acceptance test.

Completed: marketplace and payment safety

Web3 marketplace and auctions

Added or repaired fixed-price listing/cancellation/purchase controls; listing-price decoding; network, token balance, allowance and gas checks; fresh listing and entitlement checks before paying; auction creation, bidding, countdown, finalization and outbid-refund controls. Contract tests were corrected to assert buyer entitlement rather than NFT transfer.

Preserved the existing KnowledgeAsset deployment and buyer access. Optional legacy-marketplace reads, old-listing cancellation and historical marketplace selection were added so a future marketplace change does not silently discard existing entitlements. No migration was performed.

Durable Web2 payment lifecycle

Replaced in-memory-only orders with PostgreSQL payment records, unique provider references, timestamps and repeat-safe confirmation. Provider confirmation does not grant access. Mock orders cannot spend tokens or grant entitlements.

Added Razorpay test-mode checkout support with HMAC/raw-webhook verification and server checks for captured status, order, amount and currency. An authorized but uncaptured payment remains pending. Static INR test pricing is not real INR-to-USDC conversion.

Settlement source and recovery

Implemented funded **buyAssetFor** source: the payer supplies the full token payment and the designated buyer receives access. Backend settlement checkpoints the signed transaction before broadcast, serializes signer use and retries the same transaction after interruption. Canonical receipt checks and confirmation depth guard against stale/reorganized receipts.

Deployment limit: the new funded-settlement source is not deployed to the existing Amoy marketplace. Compiling or testing it locally does not enable it on that deployment.

Added bounded retries, review holds and operator recovery/full-refund verification. This is not automatic refund submission, an exclusive listing reservation, or a complete captured-payment reconciliation service.

Fee handling already established

The existing DKM platform cut is 5% of the asset fee, paid from seller proceeds. It must not be added again as a buyer surcharge. Provider/FX/network fees need a clear payer allocation and an exact, expiring quote; totals already containing a fee must not have that fee added twice.

Transak: what works and what does not

Implemented / verified	Limit still present
Staging credentials authenticate; real BUY widget session creation succeeded.	Creating a session is not completing a payment or purchasing a DKM asset.
Server-only credentials; encrypted shared access-token cache and serialized use.	No live-money configuration or production readiness is claimed.
Funding entry requires login and a verified destination wallet.	No walletless embedded-wallet journey yet.
Saved account-scoped order history; server-side order refresh validates ID, wallet, BUY direction, Polygon and USDC.	Manual order ID entry; no automatic webhook discovery. History is the 50 most recently checked saved orders.
Provider-reported status, amounts and fees shown without duplicate fee addition.	Provider report is not independent chain confirmation, an asset quote or access entitlement.
Saved seller preference: USDC, USD, EUR, INR or IDR. SELL sandbox session creation for EUR succeeded.	Preferences do not change listing prices or guarantee available payout routes. No completed withdrawal or automatic seller payout.

Why funding is still separate from a marketplace purchase

Transak staging delivers its sandbox TRNSK token, not the DKM MockUSDC used by the existing marketplace. A sandbox success therefore cannot truthfully be treated as funding the current asset purchase. The connection between payment, seller payout, blockchain receipt and access still needs implementation.

UPI, other crypto and arbitrary currencies

UPI has not been verified in the current staging path. No arbitrary-token swap/bridge route is implemented. LI.FI was investigated, but there is no supported route into this Amoy custom-token setup. Provider support, network liquidity, country and payout eligibility constrain what can be offered.

The product should show only supported combinations. It cannot promise every cryptocurrency, every payment method or every seller fiat currency. No additional Onramp.money, MoonPay or Alchemy Pay account is required by the current owner checklist.

Provider documentation used during the payment work is listed on page 11. Availability statements here reflect the recorded integration checks, not a new provider eligibility assessment on the report date.

New blockchain payment receipt registry

Latest completed work: commit 6612caf

Added **PaymentReceiptRegistry** as a separate contract. It records the authorized backend's attestation of a payment and seller payout, without replacing KnowledgeAsset or Marketplace.

Recorded data. Asset/token identity, buyer/seller wallets, separate buyer-payment and seller-payout denominations and integer amounts, a provider-reference commitment and evidence hash.

Safety controls. Authorized recorder roles, admin-controlled signer rotation, immutable sandbox classification, duplicate internal/provider-reference rejection and immutable original payment hashes.

Reversals. Full refund and chargeback records preserve the original payment hash and are terminal. Partial refunds and dispute-resolution transitions require a separate design.

Validation. Seven new local contract tests cover authorization, replay, validation, denomination separation, hashing, reversal behavior and role rotation. The full contract suite reached 128 passing tests.

What this contract does not do

It does not move money, verify a bank payment by itself, convert currencies, pay a seller or grant download access. An on-chain attestation proves that the authorized recorder made a statement; it is not independent proof that a bank transferred funds.

Connections still to build

Persist the agreed seller terms and expiring buyer quote; verify both payment and actual seller payout; store an immutable normalized receipt and durable recording job; sign with a dedicated recorder; checkpoint and retry transactions safely; verify canonical receipts/events; expose separate verified/pending/recorded UI states; connect a reviewed entitlement flow and verified reversals.

No registry deployment, public chain write, recording API, backend worker or entitlement integration is claimed. Existing wallet purchase events remain the evidence for existing on-chain marketplace purchases.

Privacy requirement

Use private salted evidence commitments and deterministic keyed provider-reference commitments. Do not put email, bank/card information, raw provider identifiers or secrets on-chain. Public wallet identifiers and asset references remain public.

Testing and security: evidence, not assumptions

Evidence recorded	Result and scope
Contracts - latest run	128 local Hardhat tests passed, including 7 receipt-registry tests. Compilation and TypeScript checking passed. Existing coverage includes minting, auctions, royalties and funded settlement.
Backend - selected suites	39 combined checks passed during order-history work; a later selection of 37 auth/payment/Transak checks passed during seller-payout work. These are different selections, not additive counts or a full production test claim.
Frontend	Production build passed. Earlier 8 regression groups and later targeted Transak/payout groups passed. Not all possible browser journeys were exercised.
Provider APIs	Real Transak staging authentication, BUY session and EUR SELL session creation succeeded. No completed real provider checkout or withdrawal was verified.
Browser / deployment	Welcome rendering and protected-page redirect checked; earlier marketplace rendering checked. Interactive buyer/seller wallet signing and full payment-to-access acceptance remain.
Static contract audit	Earlier Slither result: 0 high, 0 medium, 4 low timestamp findings and 1 compiler informational finding. New receipt registry has not been separately Slither-reviewed.

Implemented security measures

Sessions, origin/CSRF checks, CORS configuration, signature verification, auth rate limiting, payment idempotency/concurrency controls, AES-GCM content protection, wallet proof expiry/replay protection, secret-safe provider errors and canonical transaction receipt checks.

Remaining security and reliability work

Abuse/load testing; production logging review; documented and rehearsed secret rotation; backup restore drill; production key management and signer protection; timestamp warning review; Solidity compiler known-bug review and a tested upgrade decision from 0.8.20. Independent security review is still absent.

For the local college demo, test isolation, backups, replay protection and failure handling remain relevant. Mainnet security audit, production operations and live-money readiness are separate future milestones. The existing content master key must be preserved to keep encrypted content readable.

Engineering still required to finish

These are implementation tasks for me, not items that become complete merely when you provide credentials. Work should proceed in this order, with supported routes agreed before promising them.

Priority / work	Completion condition
1. Supported payment matrix	Define demonstrable buyer methods, chains/tokens, seller denominations and provider routes. Explicitly mark unavailable UPI/crypto combinations.
2. Pricing and all-in quote	Separate listing price currency from payout preference. Persist an expiring quote with integer units, FX, seller fee, external charges, net payout and expiry/insufficient-output behavior.
3. Fiat purchase and payout	Tie a provider order to the asset purchase; independently verify capture and seller payout; handle failure between those steps. Funding or a SELL session alone must not unlock content.
4. Crypto routing	Implement and verify a supported swap/bridge execution route, destination purchase and failure recovery. Requires a compatible network/token route, not a fictitious Amoy bridge.
5. Walletless buyer flow	Integrate embedded wallet/account binding and recovery after development access is ready; decide transaction sponsorship if required.
6. Blockchain recording/access	Implement verified receipt storage, durable recording jobs, dedicated signer, canonical event verification and UI states. Review and approve any necessary test-contract deployment; preserve existing assets/access.
7. Refunds and sale races	Implement provider reconciliation, retry/timeout policy, reservation or safe sale-race handling, verified refunds/chargebacks and their access consequences.
8. Activity and acceptance	Finish indexed auction/history/net earnings; run the complete buyer/seller acceptance matrix and fix discovered issues. Assemble reproducible demo evidence.

No completion percentage or delivery date is asserted: route feasibility, interactive provider access and the college rubric affect scope. A working account/API key does not resolve these engineering items.

What you need to do, step by step

1. Send the college requirements. Provide the deadline and rubric. Confirm whether QAOA pricing, VQC embeddings and rich quality scoring are mandatory, and whether a clearly labelled provider sandbox is acceptable for payment demonstrations.

2. Prepare buyer and seller accounts. Use two separate DKM accounts/browser profiles and two wallets. For the existing marketplace demo, confirm Polygon Amoy, chain ID 80002, and obtain test POL plus DKM MockUSDC. Verify each wallet on Account. Send only public addresses and buyer/seller labels.

3. Complete one Transak sandbox order. Keep partner fee at zero. Log in, verify the wallet, open Add USDC and the sandbox. Follow the official sandbox instructions only. Return to DKM and enter the order ID under Verify sandbox order. Send the order ID, payment method and any non-secret error. If UPI is absent, report it rather than making a real payment.

4. Check withdrawal eligibility if needed. Enable SELL in the Transak portal if required, save the preferred payout currency in Account and open the withdrawal sandbox. Use documented test details only. Report available routes; a saved currency preference does not prove payout eligibility.

5. Configure Google login if it is part of the demo. Create a Google OAuth Web Application client. Register http://localhost:8000/api/auth/google/callback. Save GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET and GOOGLE_REDIRECT_URI in ignored backend/.env; keep FRONTEND_ORIGIN=http://localhost:3000. Tell me configuration is ready, without sharing secrets.

6. Enable the walletless development dependency. For the no-MetaMask requirement, create a Privy development app, allow localhost and provide its public app ID. Report any setup blocker before buying a plan. Secure integration and recovery remain my work.

7. Protect project data. Back up the existing CONTENT_MASTER_KEY securely and back up the Docker PostgreSQL database. Rotate the previously exposed RPC credential in its provider dashboard and update local ignored configuration. Do not replace the content key without migration.

8. Participate in final acceptance. Keep MetaMask available for wallet signatures and test transactions. Review any concrete test deployment proposal when ready. No production hosting, real funds or extra provider signup is needed now.

Already resolved: Hardhat and its skill were found in the contract project. You do not need to install or supply Hardhat again. Transak credentials are already saved and authenticated.

Completion criteria and deferred scope

Final local acceptance run

Journey	Required evidence
Account isolation	Signup/login/logout; protected URLs; two-user isolation; wallet proof expiry/replay; Google login if configured.
Create and discover	Analyze/upload/mint/link; duplicate rejection; readable marketplace/search; creator download; unauthorized download rejected.
Fixed-price purchase	Fresh price, correct network/balance/approval, signed purchase, canonical receipt, seller/royalty/platform accounting and buyer access; creator still owns NFT.
Auction	Create, competing bids, outbid refund, expiry/finalization, correct access and payment accounting; reload/history behavior.
Supported alternative payments	Reviewed exact quote; verified provider payment and actual payout; blockchain receipt; correct access. Unsupported routes explicitly unavailable.
Failure and recovery	Duplicate/concurrent callbacks, restart after signing, stale/reorganized receipts, expired quote, cancelled listing, provider failure, refund/chargeback and no mock entitlement.

Conditional research: not implemented as requested experiments

Rich quality scoring: coherence, completeness, originality and review summaries. Existing LLM scoring is not evidence of these separate metrics.
QAOA pricing: optimizer, classical baseline, benchmark, numerical results and report section.
VQC embeddings: 384D-to-quantum/hybrid representation, training, NDCG@10 and classical comparison. Existing graph/QUBO reranking is classical and does not complete these experiments.

Lit-style decentralized key release and Kyber/post-quantum encryption also remain unimplemented roadmap ideas. Current protection uses server-managed AES-GCM. Their inclusion should follow the agreed rubric, not be claimed from unrelated code.

Outside the current local-demo delivery

Public hosting/domain/HTTPS operations; mainnet rollout; live-money merchant/KYB eligibility; regulated payout/FX operations; production treasury decisions; independent production audit and operational key/signer controls. These are not completed, but need not block a correctly scoped college sandbox demonstration.

Finish line: all agreed local requirements implemented, all required acceptance journeys passing, limitations visible in the UI/report, and final screenshots, transaction evidence and demonstration instructions prepared. This PDF is a status report, not a declaration that the application is finished.

Change history and evidence references

Date / commit	Delivered change
30 Sep / 9c58e61	Persistent test payments, funded settlement source and marketplace controls.
30 Sep / 06b1222	Account authentication and backend features connected to UI.
30 Sep / a665b7e	Marketplace navigation hidden on authentication pages.
30 Sep / 195ad63	Google login implementation and readable marketplace descriptions.
30 Sep / e117c79	Core roadmap repairs and protected marketplace/content access.
30 Sep / 0cfd760	Reviewed recovery flow and production on-ramp plan.
30 Sep / 9d2c57a	Funding readiness UI and safer wallet checks.
01 Oct / 9ba813b	Welcome/authentication gate for application UI.
01 Oct / d55cb69	Supported crypto funding guidance and testnet limitations.
09 Oct / 2110dd9	Payment total review and refreshed checkout state.
09 Oct / 0c50cee	Legacy marketplace access and canonical settlement receipts.
10 Oct / eb343cf	Authenticated Transak staging funding sessions.
10 Oct / 5ea58c1	Account-scoped verified Transak sandbox order history.
10 Oct / c2eb3d0	Seller payout preferences and hosted SELL sandbox entry.
10 Oct / 6612caf	Provider-independent payment receipt registry and tests.

Repository evidence: original pasted architecture review; PROJECT_CONTEXT.md (historical baseline); ROADMAP_STATUS.md (historical roadmap); MASTER_TODO.md; LOCAL_PAYMENT_DEMO.md; PAYMENT_RECORDING.md; commit history and prior test records. Newer documents supersede older unfinished-item lists.

Review: github.com/parzival787/capstone-project/pull/1 (existing draft PR; no merge performed).

Provider references used in the integration work: Transak sandbox instructions; widget session API; order lookup API; widget parameters. Provider eligibility must be checked for the selected route before acceptance.